University of BRISTOL

LSDE Workshop

Week 5

Dr Wei Wei

Dr Hugo Alcaraz Herrera

bristol.ac.uk

Week 6-7 Tasks

- Week 6 is Reading Week. We don’t have any live session.

- Cloud Foundations Course:
  - Module 2 (Cloud Economics & Billing).
  - Module 10 (Auto Scaling & Monitoring) *[includes Lab 6].*

- Support:
  - Office Hours Booking.
  - BB Discussion.
  - Feedback Form by 12pm.

Review of Cloud Concepts (Modules 7, 8)

- What do I actually need to learn?
  - Module 7 – Storage
    - Understand the differences between ‘object’ storage and ‘block’ or ‘file’ storage.
    - Differentiate between Amazon EBS, Amazon S3, Amazon EFS, and Amazon S3 Glacier.
    - Get some hands-on experience with S3 buckets, uploading/downloading files, etc.

Comparison of different types of storage

- Block storage:
  - Splits files into fixed size chunks of data → blocks
  - Optimal for low-latency transactional operations, I/O-intensive workloads.
- Object storage:
  - Files are stored using a flat structure in buckets.
  - Good for large and unstructured data sets.
- File storage:
  - Files are organized in a tree-like hierarchy.
  - Ideal when files are shared and managed by multiple hosts.

Choosing the Right Storage Service in AWS

Scenario 1:

You're a developer and you plan to build out an application to transcode large media files like videos. You will be using an AWS Lambda function to perform the transcoding, but you need a place to store both the original media files and the transcoded media files. Due to regulations, you need to store these files for at least a year. Which of the storage services that we've talked about in this course should you use?

Choosing the Right Storage Service in AWS

Scenario 2:

You're an architect for an ecommerce company that wants to run their MySQL database on an EC2 instance. This database needs a storage layer to store their order and customer information. The database will frequently be accessed and updated, so the storage layer needs to respond quickly. It's important that the storage is fast and durable. Which AWS storage service should you use?

Choosing the Right Storage Service in AWS

Scenario 3:

You're creating an application on multiple instances. By default, this application stores user uploads on the local file system. Since you want to use multiple instances, you will need to move this application installation and all of the user customizations into a shared storage platform. Which storage option would you use?

Review of Cloud Concepts (Modules 7, 8)

- What do I actually need to learn?
  - Module 8 – Databases
    - Understand some basic differences between ‘Relational’ (SQL) and ‘Non-relational’ (NoSQL) databases.
    - Identify the benefits & drawbacks of using RDS vs managing a database yourself.
    - Get some hands-on experience with RDS tables, adding/removing Items, etc.

Relational versus non-relational databases

	Relational (SQL)	Non-Relational
Data Storage	Rows and columns	Key-value, document, graph
Schemas	Fixed	Dynamic
Querying	Uses SQL	Focuses on collection of documents
Scalability	Vertical	Horizontal

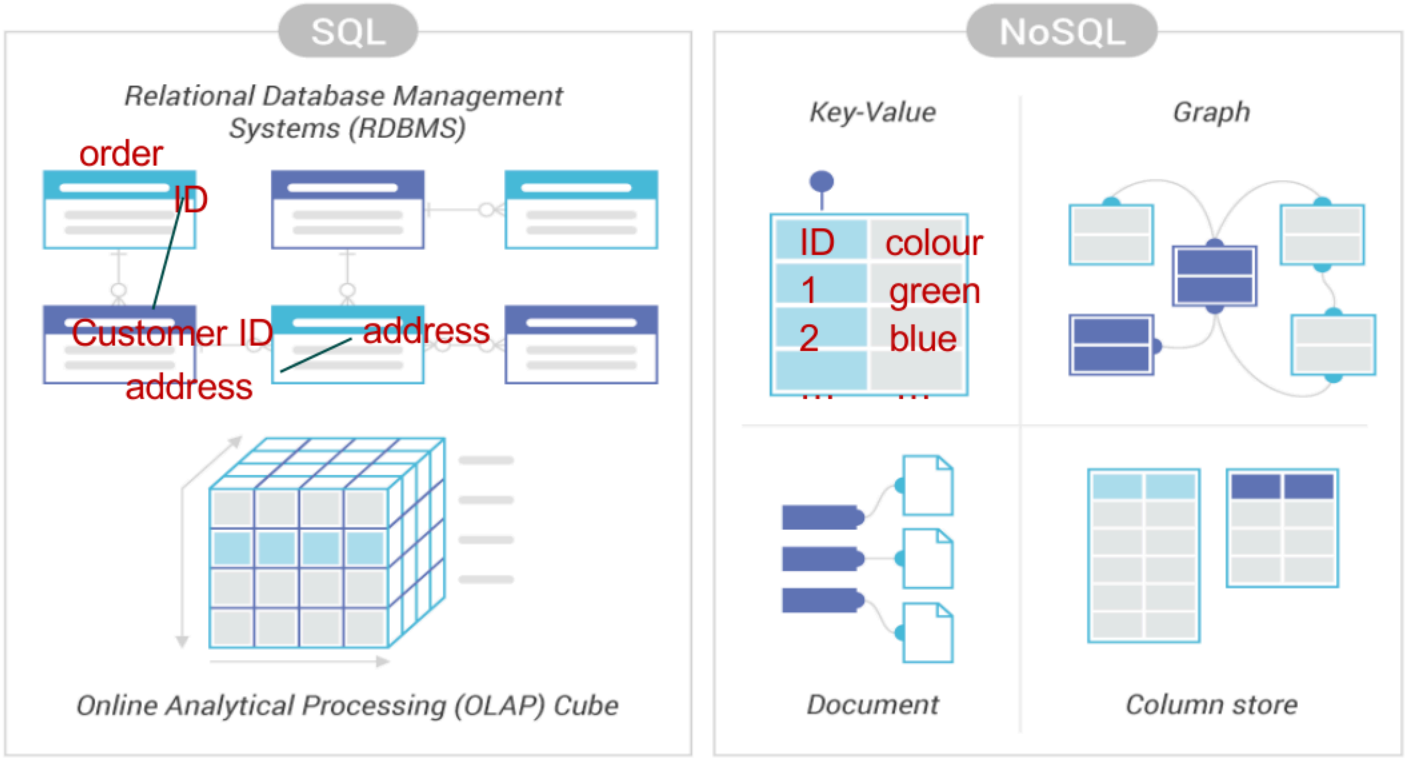
Example

ISBN	Title	Author	Format
3111111223439	Withering Depths	Jackson, Mateo	Paperback
3122222223439	Wily Willy	Wang, Xiulan	Ebook

```
{
 ISBN: 3111111223439,
 Title: "Withering Depths",
 Author: "Jackson, Mateo",
 Format: "Paperback"
}
```


AWS Slide

Relational versus non-relational databases



Some applications

- **Relational Database (RDS)**
  - Banking Transactions records:
    Tables: Customers, Accounts, Transactions.
    Query: “Total deposits by region and date”.
  - Economic Research:
    Structured datasets (GDP, CPI, trade data).
    Easy to join, aggregate, and compare.

- **Non-Relational (DynamoDB)**
  - Fintech Apps:
    Store user activity logs, clickstream data, or mobile usage patterns (JSON documents)
  - Market Sentiment Analysis:
    Collect social media posts, news feeds, and comments — flexible, fast-growing data.
  - Graph Databases:
    Model money flow networks or fraud detection using relationships between accounts.

Why databases are necessary? — Even when Python is powerful.

- Databases are essential for **scalability, reliability, and collaboration**.

Limitation in Python	**What Databases Provide**
In-memory only → crashes on big data	Scalable storage (GB–PB)
One user at a time	Multi-user concurrency (safe transactions)
No schema enforcement	Data integrity & relationships & flexible schema
Manual file I/O	Persistent, queryable storage
Basic security	Authentication, encryption, audit logs
Slow for large joins	Indexed, optimized queries

University of BRISTOL

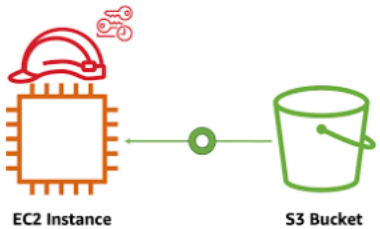
S3

Activity

bristol.ac.uk

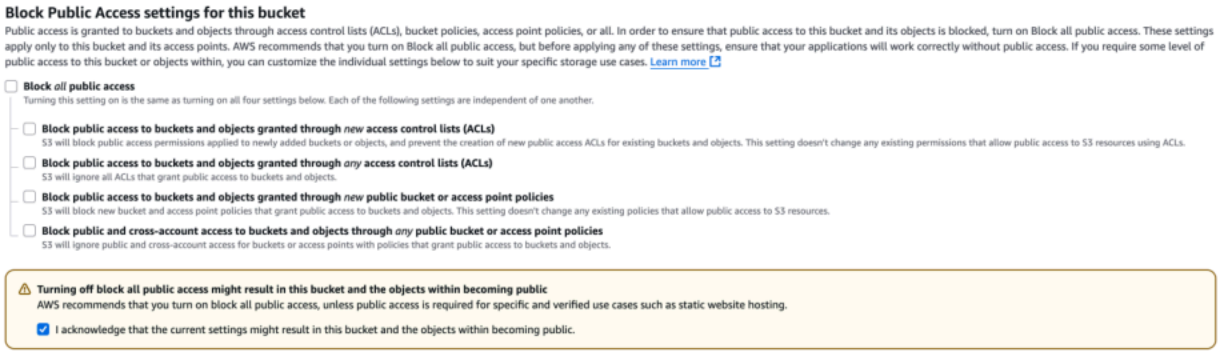
Activity – S3 buckets

- Activity introduction:
  - In the AWS Academy course Sandbox, we will set up an EC2 instance with SSH access, and S3 bucket. Know how to use EC2 instance to access the files saved in S3 bucket.
  - We will learn:
    - Obtain **bristol.txt** data (available on BB) and upload it to S3.
    - Host a static website using S3 bucket.
    - Use the AWS CLI to copy the data from S3 to the EC2 instance.
    - Use the AWS CLI to copy the data from EC2 instance to S3 bucket.



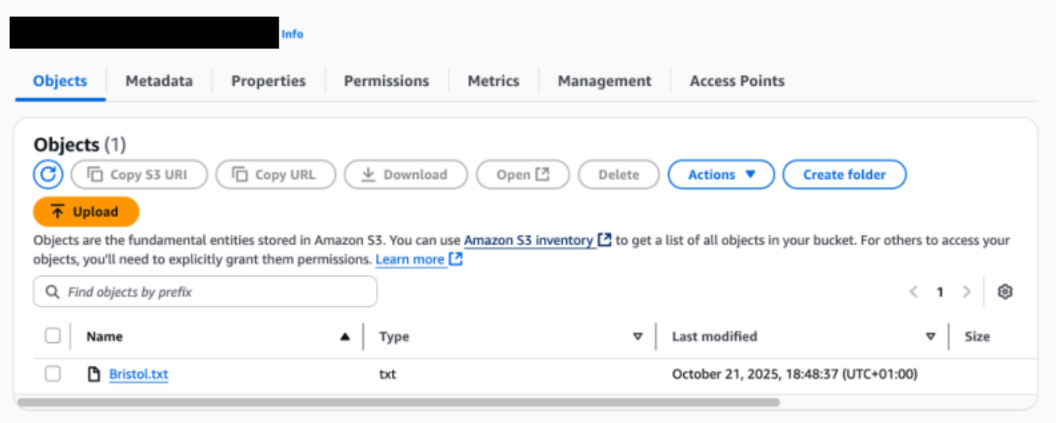
Activity – S3 buckets (Host a static website)

- Step1: Create an S3 buckets and configure settings.
  - Create an S3 bucket:
    - Bucket type: General purpose.
    - You should give a globally unique name for your bucket.
    - Object ownership: ACLs enabled, Bucket owner preferred.
    - Untick Block Public Access settings for this bucket and tick “I acknowledge that …”
    - Click create bucket.



Activity – S3 buckets (Host a static website)

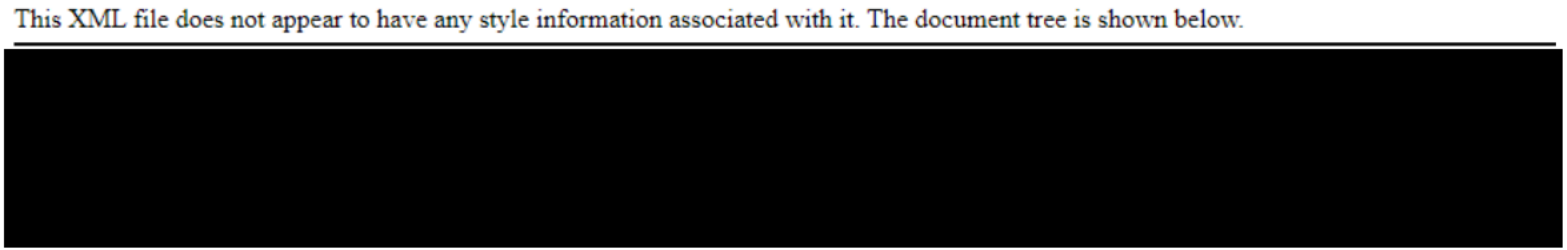
- Step 2: Upload file in this bucket.
  - Download Bristol.txt from Blackboard.
  - Open the bucket you just created and click upload.
  - Drag Bristol.txt file into the bucket, click upload.
  - You will see Bristol.txt is listed in the bucket.



Activity: S3 buckets (Publicly Accessible)

- Step 3: Configure your file.
  - Check whether you can access to this file via Internet.

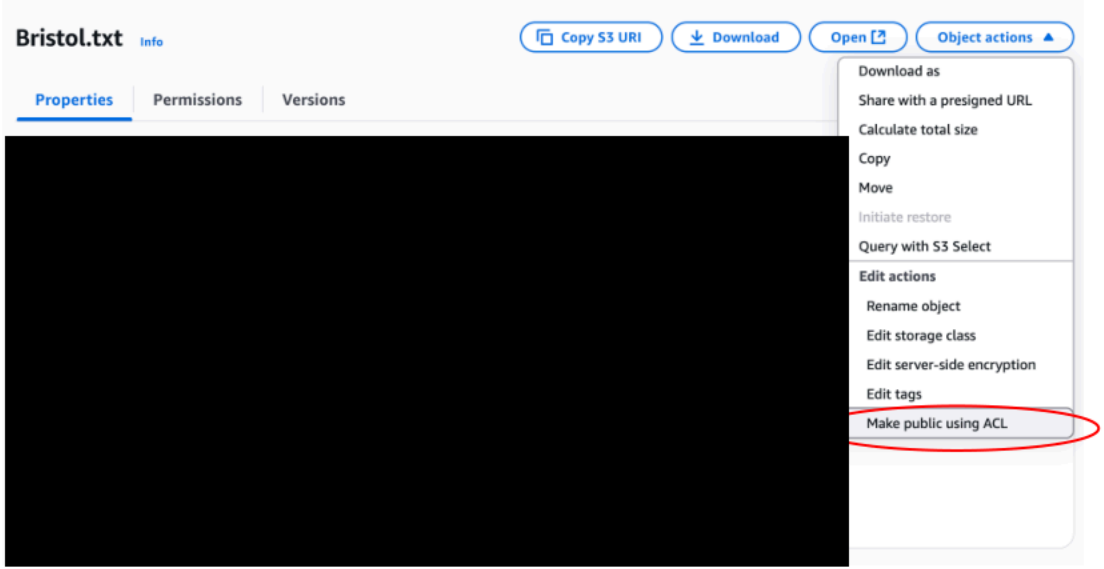
    Copy the URL of Bristol.txt in S3 bucket and paste it in a browser:



Hint: Is the S3 bucket publicly accessible? Yes, but the file is not public accessible.

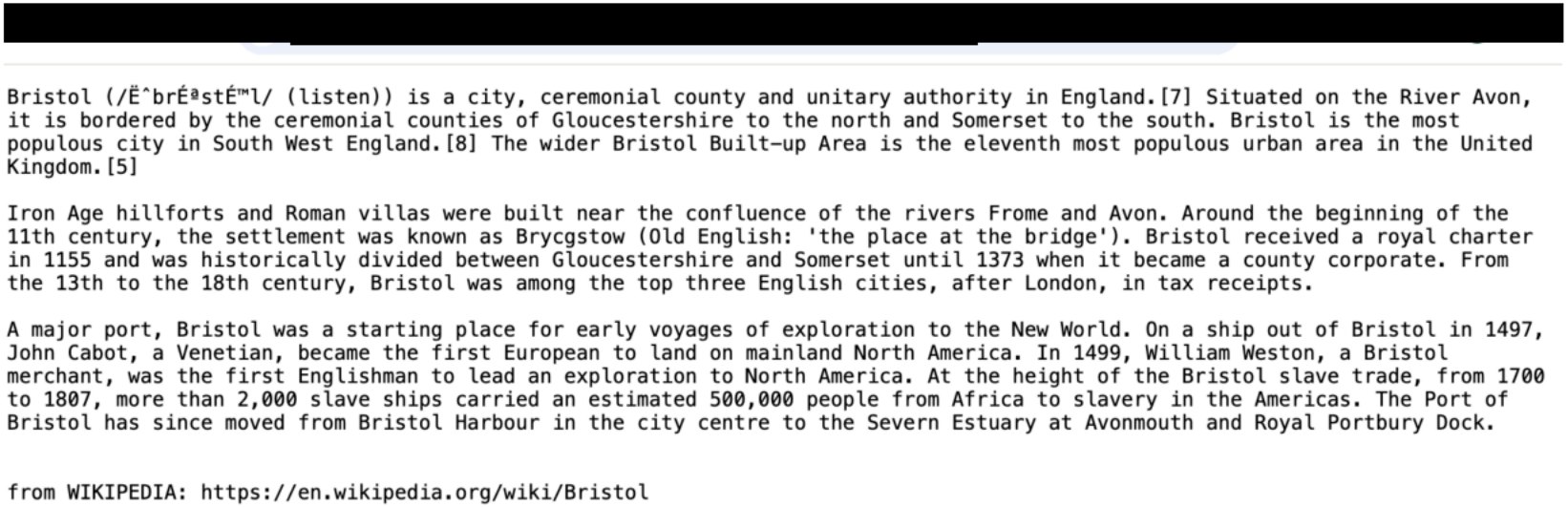
Activity: S3 buckets (Publicly Accessible)

- Click object Bristol.txt.
- Click Object actions -> Make public using ACL, click make public.



Activity: S3 buckets (Publicly Accessible)

– Refresh your browser and now you could access your website successfully:



Bristol (/ËˆbrÉªstÉ™l/ (listen)) is a city, ceremonial county and unitary authority in England.[7] Situated on the River Avon, it is bordered by the ceremonial counties of Gloucestershire to the north and Somerset to the south. Bristol is the most populous city in South West England.[8] The wider Bristol Built-up Area is the eleventh most populous urban area in the United Kingdom.[5]

Iron Age hillforts and Roman villas were built near the confluence of the rivers Frome and Avon. Around the beginning of the 11th century, the settlement was known as Brycgstow (Old English: 'the place at the bridge'). Bristol received a royal charter in 1155 and was historically divided between Gloucestershire and Somerset until 1373 when it became a county corporate. From the 13th to the 18th century, Bristol was among the top three English cities, after London, in tax receipts.

A major port, Bristol was a starting place for early voyages of exploration to the New World. On a ship out of Bristol in 1497, John Cabot, a Venetian, became the first European to land on mainland North America. In 1499, William Weston, a Bristol merchant, was the first Englishman to lead an exploration to North America. At the height of the Bristol slave trade, from 1700 to 1807, more than 2,000 slave ships carried an estimated 500,000 people from Africa to slavery in the Americas. The Port of Bristol has since moved from Bristol Harbour in the city centre to the Severn Estuary at Avonmouth and Royal Portbury Dock.

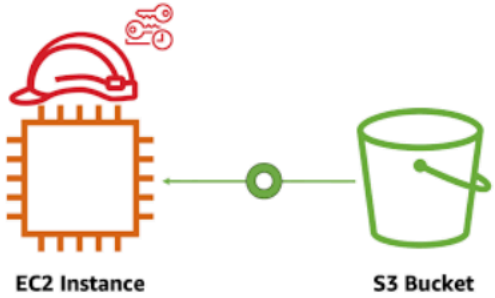
from WIKIPEDIA: https://en.wikipedia.org/wiki/Bristol

Activity: S3 buckets

- Step 4: Launch an EC2 instance and connect via SSH.
  - Set this EC2 instance with the following main properties:
    - AMI: Amazon Linux 2023 AMI.
    - Instance size: t3.micro (free tier).
    - EC2 Keypair: Create a new keypair and save the file.
    - Security Group: SSH access from anywhere.
  - Connect via SSH (using a PuTTy or command line client).

If you are not sure about SSH connection, please visit Video demos on BB (week 4).

Activity: S3 buckets



- Step 4: Manage files in S3 using SSH connection.
  - List your S3 buckets:

```
aws s3 ls
```

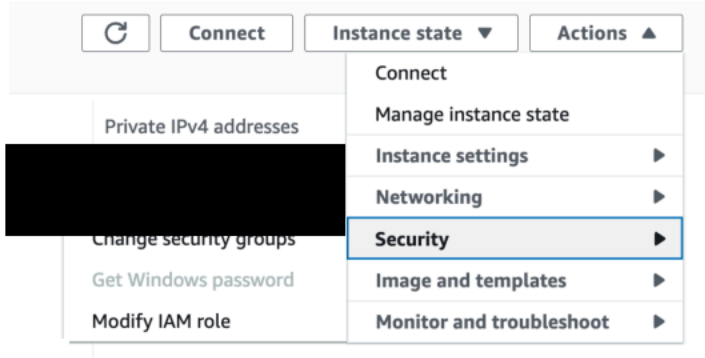
```
[redacted]~]$ aws s3 ls
Unable to locate credentials. You can configure credentials by running "aws conf
igure".
[redacted]~]$
```

  - It will show that we failed to list S3 buckets .

  - Question: Why are we unable to access S3 buckets?
  - Hint: IAM roles.

Activity: S3 buckets

– Attach an IAM Role to your instance:
  - Select your instance, click: Actions→Security→Modify IAM role.
  - Select EMR_EC2_DefaultRole.



– Try to list S3 buckets again:

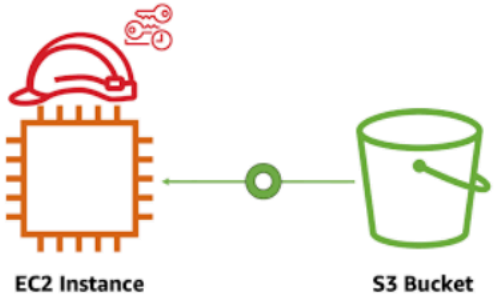
```
[redacted]~]$ aws s3 ls
2025-10-21 17:46:43 [redacted]
[redacted]~]$
```

– List all files in your S3 Bucket using code:

```
aws s3 ls s3://your bucket name
```

```
[redacted]~]$ aws s3 ls [redacted]
2025-10-21 17:48:37       1555 Bristol.txt
[redacted]~]$
```

Activity: S3 buckets

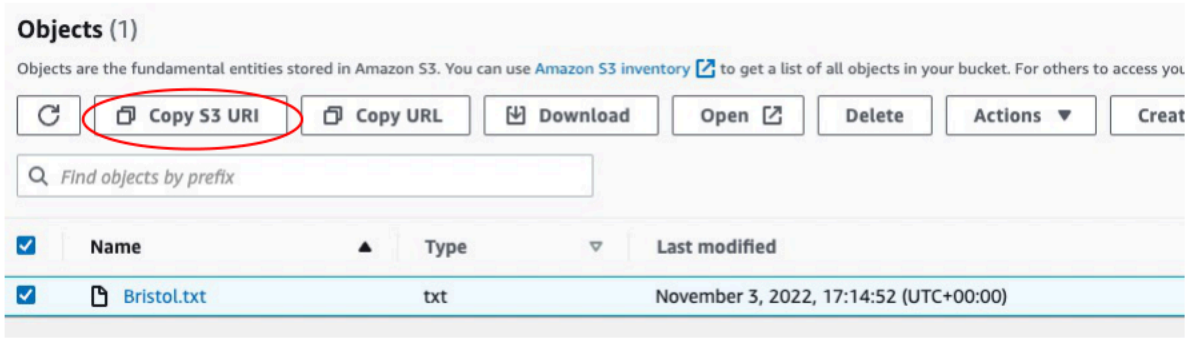


– Copy files from S3 to your EC2 instance:

```
aws s3 cp <Your S3_URI> .
```

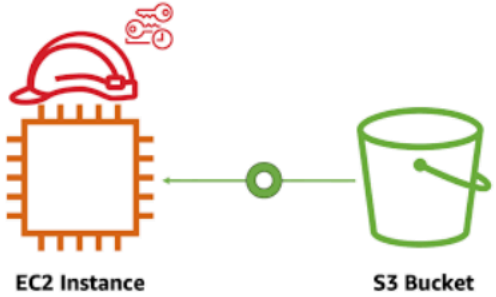
Don't forget the final dot, indicating to copy to the current directory.

```
[redacted] ~]$ aws s3 cp [redacted] .
download: [redacted] to ./Bristol.txt
[redacted] ~]$
```



Hint: how to get your S3 URI.

Activity: S3 buckets



– Create another copy of Bristol.txt to the current destination(EC2 instance):

```
cp Bristol.txt Bristol_copy.txt
```

```
[                                ~]$ cp Bristol.txt Bristol_copy.txt
[                                ~]$ ls
Bristol.txt  Bristol_copy.txt
```

We use *ls* to check what are listed in the current folder.

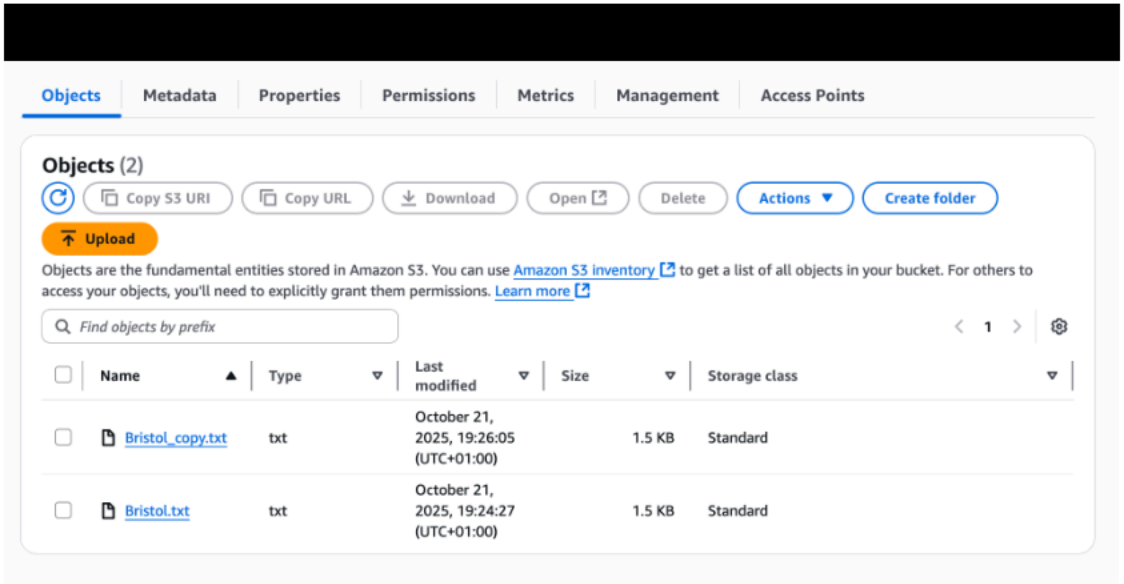
– Copy Bristol_copy.txt to your S3 bucket and list the files in your S3 bucket:

```
aws s3 cp Bristol_copy.txt s3:// your bucket name
aws s3 ls s3://your bucket name
```

```
[                                ~]$ aws s3 cp Bristol_copy.txt
upload: ./Bristol_copy.txt to
[                                ~]$ aws s3 ls
2025-10-21 18:24:27       1555 Bristol.txt
2025-10-21 18:26:05       1555 Bristol_copy.txt
                                 ~]$
```

Activity: S3 buckets

– Check your S3 bucket on AWS console. Now you should be able to see Bristol_copy.txt in your bucket:



Activity: S3 buckets (Challenge question)

Question: How to empty your S3 bucket with AWS CLI?

Try to use the code below and check your S3 bucket:

```
aws s3 rm s3:// your bucket-name --recursive
```

University of BRISTOL

RDS

Challenge Activity

bristol.ac.uk

Activity – RDS

- Activity introduction:
  - In the AWS Academy course Sandbox, we will set up an EC2 instance with SSH access, an S3 bucket and a RDS database.
  - We will:
    - Obtain some CSV finance data and upload it to S3.
    - Use the AWS CLI to copy the data from S3 to the EC2 instance.
    - Use the AWS CLI to import the CSV data to RDB.
    - Access and modify table data via the AWS Console, CLI and using Python.
  - The steps 2-4 are similar to some process in the activity of S3.

Activity – RDS

- Step 1: Prepare your data and materials.
  - Download AMZN.csv from the LSDE BB:

    (Or you can download the Amazon.com share data via the link. On the Historical Data tab click Download to obtain the CSV file and review in Excel or similar spreadsheet/editor. You may choose a difference company!).

	A	B	C	D	E	F	G
1	Date	Open	High	Low	Close	Adj Close	Volume
2	09/11/2020	3231.03003	3289	3112.11011	3143.73999	3143.73999	7190400
3	10/11/2020	3095.02002	3114	3019.47998	3035.02002	3035.02002	6591000
4	11/11/2020	3061.78003	3139.1499	3050	3137.38989	3137.38989	4366900
5	12/11/2020	3159.94995	3175.87988	3086.05005	3110.28003	3110.28003	4362000
6	13/11/2020	3122	3141.71997	3085.38989	3128.81006	3128.81006	3756200
7	16/11/2020	3093.19995	3142.69995	3072.68994	3131.06006	3131.06006	3808700
8	17/11/2020	3183.54004	3189.25	3135.26001	3135.65991	3135.65991	3444700
9	18/11/2020	3134	3140	3105.1001	3105.45996	3105.45996	2916800

Activity – RDS

- Step 2: Create an S3 bucket and upload data.
  - Create an S3 bucket with unique name.
  - Configure your S3 buckets:
    - Untick Block all public access.
    - Select ACLs enabled.
  - Upload data on the bucked you created.
  - Make the file public using ACL.
  - If your setting is correct, using object URL can allow to download this file.

Activity – RDS

- Step 3: Launch an EC2 instance and connect it via SSH.
  - The instance should have the following main properties:
    - AMI: Amazon Linux 2.
    - Instance size: T3.micro (free tier).
    - EC2 Keypair: Create a new keypair after pressing Launch and save the file.
      Mac: download .pem; Windows: download .ppk
    - Security Group: SSH (port 22) access.
  - Grant an IAM Role (EMR_EC2_DefaultRole) to this instance.
  - Connect the instance via SSH.

Activity – RDS

- Step 4: Copy the data file on EC2 instance from S3 buckets.
  - Copy your CSV URI from the S3 bucket (repeat for each copied S3 file URI):

```
aws s3 cp <S3 URI> .
```

Here I use the bucket in S3 activity.



  - Quit your SSH, by input “exit”.



Activity – RDS

- Step 5: Create an RDS DB instance on AWS console.
  - On the Services menu, choose **Aurora & RDS**.
  - Click Create database.
  - Main properties: *(You can review Lab 5 to know how to set properties)*
    - **Standard create** with **MySQL** engine.
    - Templates: **Dev/Test** **Free tier**; Availability: **Single-AZ DB instance**.
    - Credential Settings: select **Self managed** credential management and set your password. Please remember **Master username** and the **Master password** you set.
    - Instance configuration: **Burstable classes**, db.t3.micro.
    - Storage: SSD(gp3), 20GiB.

NOTE: Most settings here aim to create a simple RDS for time-saving.

Activity – RDS

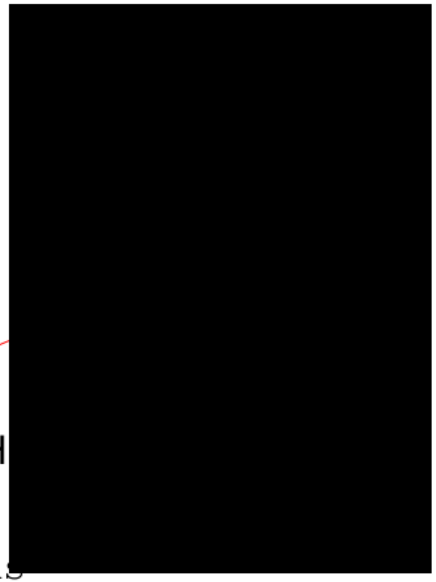
- Main properties-continue:
  - Connectivity: choose **Connect to an EC2 compute resource** and select the EC2 instances you just launched.
  - Security Group: default.
  - Monitoring: **Untick** enable enhanced monitoring.
  - Additional configuration: **Untick** automated backups and encryption.

NOTE: Most settings here aim to create a simple RDS for time-saving.

Activity – RDS

- Step 6: Connect RDS DB instance vis SSH.
  - Wait until your database is ready (this step will take long time).
  - Open your SQL DB instance and copy the **RDS endpoint**.
  - To connect to your RDS instance via SSH, you need to create an SSH
    - For Mac, input the following code:

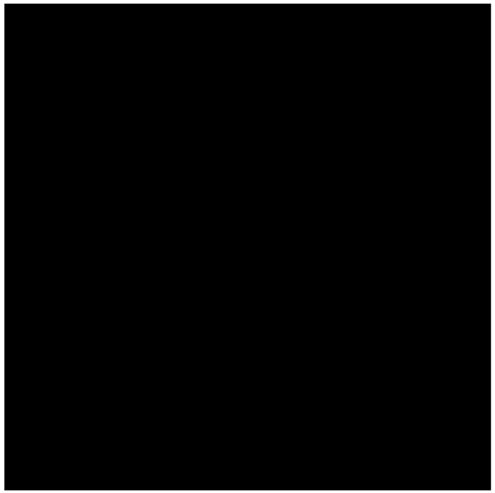
```
ssh -i [your-aws-ec2-keypair].pem -L 3306:[your-rds
endpoint]:3306 ec2-user@[your-ec2-instance-ip]
```





Activity – RDS

- For Window using PuTTy:
  - Open your PuTTy.
  - Session → Host name: ec2-user@[instance- public-DNS].
  - SSH → Auth → Credential: upload your .ppk key file.
  - SSH → Tunnels:
    Source port: 3306
    Destination: [your-rds-endpoint]:3306



Activity – RDS

If you get "No match for argument: mysql Error: Unable to find a match: mysql " when you try to install SQL client, you can use different code to install it:

```
sudo dnf update -y
sudo dnf install mariadb105
```

Please refer to this link:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_GettingStarted.CreatingConnecting.MySQL.html

– Install SQL client:

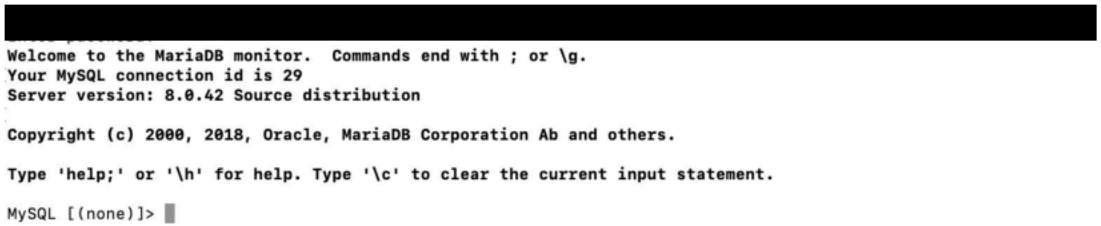
```
sudo yum update
sudo yum install mysql
```

– Access your SQL database instance:

```
mysql -h [your-rds-endpoint] -P 3306 -u [rds-master-username] -p
```

Make sure you type the command manually or ensure that the dashes are the ASCII hyphen-minus character (-), not a different Unicode dash character.

– Then enter your RDS Master password and press ENTER directly. The password **will not** show on the screen:



Activity – RDS

- Step 7: Create a database in SQL DB instance and import data from the AMZN.csv file.
  - Create a database, give a name as you like:

```
CREATE DATABASE [name_database];
```

  - Open this database:

```
USE [name_database];
```

  - Create a table in this database and define the data structure:

```
CREATE TABLE [name_table](
     date DATE PRIMARY KEY,
     open DECIMAL(10, 2) NOT NULL,
     high DECIMAL(10, 2) NOT NULL,
     low DECIMAL(10, 2) NOT NULL,
     close DECIMAL(10, 2) NOT NULL,
     adj DECIMAL(10, 2) NOT NULL,
     volume DECIMAL(10, 2) NOT NULL
);
```

Activity – RDS

– Import the CSV data from your EC2 instance into this table:

```
LOAD DATA LOCAL INFILE 'AMZN.csv'
INTO TABLE [name_table]
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(date, open, high, low, close, adj, volume);
```

```
MySQL [amazon]> LOAD DATA LOCAL INFILE 'AMZN.csv'
    -> INTO TABLE amazon_data
    -> FIELDS TERMINATED BY ',' ENCLOSED BY '"'
    -> LINES TERMINATED BY '\n'
    -> IGNORE 1 LINES
    -> (date, open, high, low, close, adj, volume);
Query OK, 251 rows affected, 990 warnings (0.041 sec)
Records: 251  Deleted: 0  Skipped: 0  Warnings: 990

MySQL [amazon]>
```

– Check the contents of the table:

```
SELECT * FROM [name_table];
```

```
MySQL [amazon]> SELECT * FROM amazon_data;
+------------+---------+---------+---------+---------+---------+-----------+
| date       | open    | high    | low     | close   | adj     | volume    |
+------------+---------+---------+---------+---------+---------+-----------+
| 2020-11-09 | 3231.03 | 3289.00 | 3112.11 | 3143.74 | 3143.74 | 7190400.00 |
| 2020-11-10 | 3095.02 | 3114.00 | 3019.48 | 3035.02 | 3035.02 | 6591000.00 |
| 2020-11-11 | 3061.78 | 3139.15 | 3050.00 | 3137.39 | 3137.39 | 4366900.00 |
| 2020-11-12 | 3159.95 | 3175.88 | 3086.05 | 3110.28 | 3110.28 | 4362000.00 |
| 2020-11-13 | 3122.00 | 3141.72 | 3085.39 | 3128.81 | 3128.81 | 3756200.00 |
| 2020-11-16 | 3093.20 | 3142.70 | 3072.69 | 3131.06 | 3131.06 | 3808700.00 |
| 2020-11-17 | 3183.54 | 3189.25 | 3135.26 | 3135.66 | 3135.66 | 3444700.00 |
| 2020-11-18 | 3134.00 | 3140.00 | 3105.10 | 3105.46 | 3105.46 | 2916800.00 |
| 2020-11-19 | 3105.31 | 3125.00 | 3080.92 | 3117.02 | 3117.02 | 3010300.00 |
| 2020-11-20 | 3117.02 | 3132.89 | 3098.05 | 3099.40 | 3099.40 | 3374400.00 |
```

Activity – RDS

- Step 8: Manage data in the database using SQL query.
  - Query data for a specific date. For example, to get the data with date ='2021-10-15':

```
SELECT * FROM [name_table]
WHERE date = '2021-10-15';
```

```
MySQL [amazon]> SELECT * FROM amazon_data
    -> WHERE date = '2021-10-15';
+------------+---------+---------+---------+---------+---------+------------+
| date       | open    | high    | low     | close   | adj     | volume     |
+------------+---------+---------+---------+---------+---------+------------+
| 2021-10-15 | 3311.42 | 3410.42 | 3304.00 | 3409.02 | 3409.02 | 5175100.00 |
+------------+---------+---------+---------+---------+---------+------------+
1 row in set (0.009 sec)
```

  - Update the data.

    For example, change the open and close columns into new values for a specific date '2021-10-15':

```
UPDATE [name_table]
SET open = 1000.00, close = 1100.00
WHERE date = '2021-10-15';
```

```
MySQL [amazon]> UPDATE amazon_data
    -> SET open = 1000.00, close = 1100.00
    -> WHERE date = '2021-10-15';
Query OK, 1 row affected (0.018 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MySQL [amazon]> SELECT * FROM amazon_data WHERE date = '2021-10-15';
+------------+---------+---------+---------+---------+---------+------------+
| date       | open    | high    | low     | close   | adj     | volume     |
+------------+---------+---------+---------+---------+---------+------------+
| 2021-10-15 | 1000.00 | 3410.42 | 3304.00 | 1100.00 | 3409.02 | 5175100.00 |
+------------+---------+---------+---------+---------+---------+------------+
1 row in set (0.002 sec)
```

Activity – RDS

- Step 7a: Create a database in SQL DB instance and import data from the Water.csv file.
  - Create a database, give a name as you like:

```
CREATE DATABASE [name_database];
```

  - Open this database:

```
USE [name_database];
```

  - Create a table in this database and define the data structure:

```
CREATE TABLE [name_table](
 date DATE PRIMARY KEY, precipitation DECIMAL(10, 2) NOT NULL,
pet DECIMAL(10, 2) NOT NULL, temperature DECIMAL(10,2) NOT NULL,
disch_spec DECIMAL(10, 2) NULL, disch_vol DECIMAL(10, 2) NULL,
peti DECIMAL(10, 2) NOT NULL, humidity DECIMAL (10,2) NOT NULL,
short_rad DECIMAL(10, 2) NOT NULL, long_rad DECIMAL(10,2) NOT NULL,
windspeed DECIMAL(10,2) NOT NULL
);
```

Activity – RDS

– Import the CSV data from your EC2 instance into this table:

```
LOAD DATA LOCAL INFILE 'Water.csv'
INTO TABLE [name_table]
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(date, precipitation, pet, temperature,
 disch_spec, disch_vol, peti, humidity
 short_rad, long_rad, windspeed);
```

```
MySQL [water]> LOAD DATA LOCAL INFILE 'Water.csv'
    -> INTO TABLE water_data
    -> FIELDS TERMINATED BY ',' ENCLOSED BY '"'
    -> LINES TERMINATED BY '\n'
    -> IGNORE 1 LINES
    -> (date, precipitation, pet, temperature, disch_spec,
    -> disch_vol, peti, humidity, short_rad, long_rad,
    -> windspeed);
Query OK, 16436 rows affected, 18340 warnings (0.396 sec)
Records: 16436  Deleted: 0  Skipped: 0  Warnings: 18340
```

– Check the contents of the table:

```
SELECT * FROM [name_table];
```

```
MySQL [water]> SELECT * FROM water_data LIMIT 30;
+------------+---------------+------+-------------+-----------+-----------+------+----------+-----------+----------+-----------+
| date       | precipitation | pet  | temperature | disch_spec | disch_vol | peti | humidity | short_rad | long_rad | windspeed |
+------------+---------------+------+-------------+-----------+-----------+------+----------+-----------+----------+-----------+
| 1970-10-01 |          9.18 | 1.00 |        9.15 |      0.00 |      0.00 | 1.27 |     6.11 |     63.83 |   325.31 |      7.30 |
| 1970-10-02 |          3.63 | 1.41 |        8.08 |      0.00 |      0.00 | 1.76 |     5.28 |     96.18 |   294.81 |     10.06 |
| 1970-10-03 |          8.64 | 1.14 |        9.08 |      0.00 |      0.00 | 1.37 |     5.57 |     63.18 |   322.15 |      5.38 |
| 1970-10-04 |          3.60 | 0.19 |       10.14 |      0.00 |      0.00 | 0.30 |     7.74 |     43.08 |   343.07 |      6.92 |
| 1970-10-05 |          0.38 | 1.56 |        9.98 |      0.00 |      0.00 | 1.85 |     5.68 |     93.72 |   300.81 |      7.65 |
| 1970-10-06 |          1.43 | 1.03 |        9.02 |      0.00 |      0.00 | 1.32 |     5.97 |     60.16 |   322.43 |      7.86 |
| 1970-10-07 |         10.66 | 0.48 |        8.30 |      0.00 |      0.00 | 0.61 |     6.24 |     58.99 |   320.15 |      3.95 |
| 1970-10-08 |          1.64 | 0.69 |        8.68 |      0.00 |      0.00 | 0.89 |     6.10 |     57.29 |   321.64 |      5.33 |
| 1970-10-09 |          0.20 | 0.84 |        8.93 |      0.00 |      0.00 | 1.12 |     6.18 |     52.91 |   325.33 |      8.08 |
| 1970-10-10 |          0.00 | 1.04 |       10.27 |      0.00 |      0.00 | 1.04 |     6.22 |     66.44 |   318.89 |      5.56 |
```

Activity – RDS

- Step 8a: Manage data in the database using SQL query.
  - Query data for a specific date. For example, to get the data with date =‘1970-10-01’:

```
SELECT * FROM [name_table]
WHERE date = '1970-10-01';
```

```
MySQL [water]> SELECT * FROM water_data
    -> WHERE date = '1970-10-01';
+------------+---------------+------+-------------+------------+----------+------+----------+-----------+----------+-----------+
| date       | precipitation | pet  | temperature | disch_spec | disch_vol | peti | humidity | short_rad | long_rad | windspeed |
+------------+---------------+------+-------------+------------+----------+------+----------+-----------+----------+-----------+
| 1970-10-01 |          9.18 | 1.00 |        9.15 |       0.00 |      0.00 | 1.27 |     6.11 |     63.83 |   325.31 |      7.30 |
+------------+---------------+------+-------------+------------+----------+------+----------+-----------+----------+-----------+
1 row in set (0.004 sec)
```

  - Update the data.

  For example, change the open and close columns into new values for a specific date ‘1970-10-01’:

```
UPDATE [name_table]
SET disch_spec = 10.00,
disch_vol = -10.00
WHERE date = '1970-10-01';
```

```
MySQL [water]> UPDATE water_data
    -> SET disch_spec = 10.00,
    -> disch_vol = -10.00
    -> WHERE date = '1970-10-01';
Query OK, 1 row affected (0.039 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MySQL [water]> SELECT * FROM water_data WHERE date = '1970-10-01';
+------------+---------------+------+-------------+------------+----------+------+----------+-----------+----------+-----------+
| date       | precipitation | pet  | temperature | disch_spec | disch_vol | peti | humidity | short_rad | long_rad | windspeed |
+------------+---------------+------+-------------+------------+----------+------+----------+-----------+----------+-----------+
| 1970-10-01 |          9.18 | 1.00 |        9.15 |      10.00 |    -10.00 | 1.27 |     6.11 |     63.83 |   325.31 |      7.30 |
+------------+---------------+------+-------------+------------+----------+------+----------+-----------+----------+-----------+
1 row in set (0.001 sec)
```